

# Institute of Information Technology

## University of Dhaka

|                     |
|---------------------|
| <b>"AI Chatbot"</b> |
|---------------------|

### **Submitted by:**

Md. Abul Kalam Azad | BSSE 1425  
Kazi Nahid Hossen Rabbi | BSSE 1437

### **Submitted to:**

Md. Saeed Siddik  
Assistant Professor

---

**Submitted on 14th May, 2026**

## 1. Project Objective

To develop a secure, full-stack AI chatbot application that provides authenticated users with access to conversational AI powered by a locally running Large Language Model (LLM) through Ollama. The system persists chat history per user in MongoDB, ensuring continuity across sessions while maintaining privacy and security through JWT-based authentication — all without relying on external AI API keys.

## 2. Core Features

- User registration and login with JWT authentication
- Interactive chat interface for sending questions and receiving AI responses
- Local LLM integration through Ollama, using the llama3 model by default
- Persistent chat history stored per user in MongoDB across sessions
- Dedicated landing page, authentication page, and chat interface with responsive UI
- Theme support for enhanced user experience
- Health check endpoint for monitoring system status

## 3. Technology Stack

| Layer          | Technology                       |
|----------------|----------------------------------|
| Frontend       | HTML5, CSS3, Vanilla JavaScript  |
| Backend        | Node.js, Express.js              |
| Database       | MongoDB with Mongoose ODM        |
| Authentication | JWT (JSON Web Tokens) and bcrypt |
| AI Model       | Ollama with Llama3 (local LLM)   |

## 4. Application Flow

1. Users open the home page to learn about the AI assistant and its capabilities.
2. New or returning users navigate to the authentication page to log in or register.
3. After successful authentication, users are redirected to the main chat interface.
4. Messages typed by the user are sent to the Express.js backend via REST API.
5. The backend validates the JWT token, then proxies the message to the local Ollama API.
6. The AI response is returned to the frontend and displayed in the chat interface. Both the user message and AI response are saved in MongoDB under the user's history.

## 5. Frontend Structure

The frontend is composed of three HTML pages and supporting JavaScript and CSS modules:

### 5.1 HTML Pages

| File              | Purpose                                         |
|-------------------|-------------------------------------------------|
| <b>home.html</b>  | Public landing page introducing the assistant   |
| <b>login.html</b> | Authentication UI for login and registration    |
| <b>index.html</b> | Main chat experience and conversation interface |

## 5.2 JavaScript Modules

| File               | Responsibility                                     |
|--------------------|----------------------------------------------------|
| <b>js/auth.js</b>  | Manages session state and backend authentication   |
| <b>js/login.js</b> | Handles login and registration form behavior       |
| <b>js/main.js</b>  | Powers the chat interface and chat history loading |

## 5.3 Stylesheets

| File                 | Scope                               |
|----------------------|-------------------------------------|
| <b>css/home.css</b>  | Visual design for the landing page  |
| <b>css/login.css</b> | Styling for the authentication page |
| <b>css/style.css</b> | Main styles for the chat interface  |

# 6. Backend Structure

The backend resides in the `server/` folder and is built with Express.js, MongoDB (via Mongoose), JWT, bcrypt, and node-fetch.

## 6.1 Main Responsibilities

- Validate and manage user registration and login
- Generate and verify JWT tokens for authenticated sessions
- Store and retrieve per-user chat history from MongoDB
- Proxy chat requests to the locally running Ollama API
- Serve frontend static files to the browser

## 6.2 API Routes

| Method      | Endpoint                        | Description                                    |
|-------------|---------------------------------|------------------------------------------------|
| <b>GET</b>  | <code>/api/health</code>        | Health check — confirms the server is running  |
| <b>POST</b> | <code>/api/auth/register</code> | Create a new user account                      |
| <b>POST</b> | <code>/api/auth/login</code>    | Authenticate a user and return a JWT           |
| <b>GET</b>  | <code>/api/auth/me</code>       | Fetch the currently authenticated user profile |

| Method        | Endpoint              | Description                                     |
|---------------|-----------------------|-------------------------------------------------|
| <b>POST</b>   | /api/chat             | Send a message to Ollama and return AI response |
| <b>GET</b>    | /api/history          | Load saved chat history for the current user    |
| <b>POST</b>   | /api/history/messages | Store a new message in chat history             |
| <b>DELETE</b> | /api/history          | Clear the current user's chat history           |

## 7. Conclusion

This project delivers a production-ready, full-stack AI chatbot that combines a modern conversational interface with secure JWT-based authentication, per-user persistent history, and a locally hosted LLM backend via Ollama. By eliminating the dependency on external AI API keys and supporting self-hosted deployment with MongoDB and Ollama, the application is particularly suitable for privacy-conscious, offline-friendly, or enterprise environments.

The modular architecture — with a clear separation between the frontend pages, JavaScript modules, and Express.js backend routes — makes the codebase maintainable and extensible. Future enhancements could include support for multiple LLM models, conversation export, role-based access control, and real-time streaming responses.